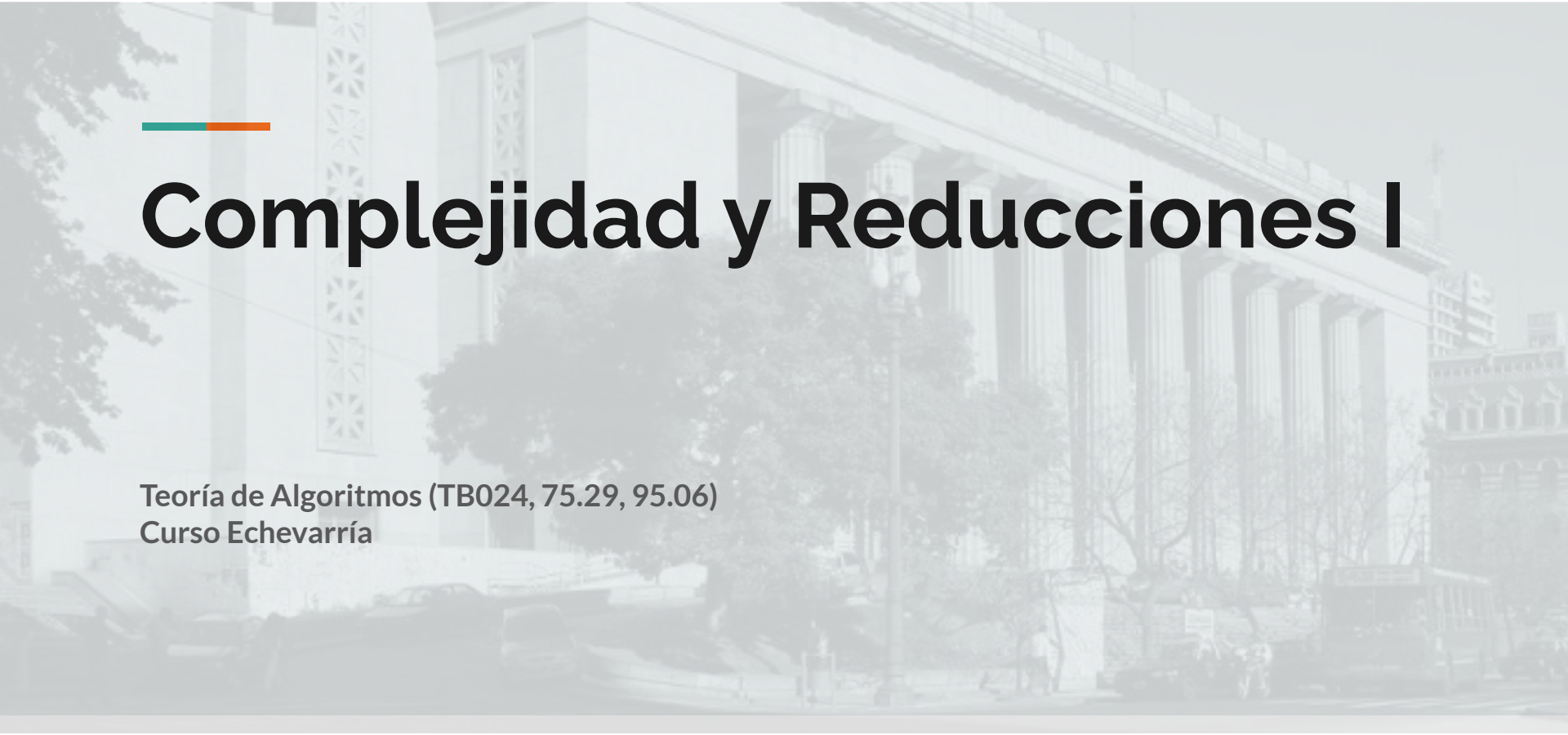


Complejidad y Reducciones I

Teoría de Algoritmos (TB024, 75.29, 95.06)
Curso Echevarría

¿Qué hace que un problema sea complejo?



Complejidad

- ¿Qué hace que un problema sea difícil? (No digan “exponencial”)
- ¿Queremos resolver cualquier problema o alguna/s instancia/s?
- ¿Queremos resolverlo?
- ¿Por qué estudiamos esto?
- ¿Dónde está mi mamá?



Complejidad

Una de las grandes preguntas que planteó David Hilbert al mundo matemático fue: Dado cualquier “problema”, ¿existe una manera de resolverlo en una serie finita de pasos ordenados?

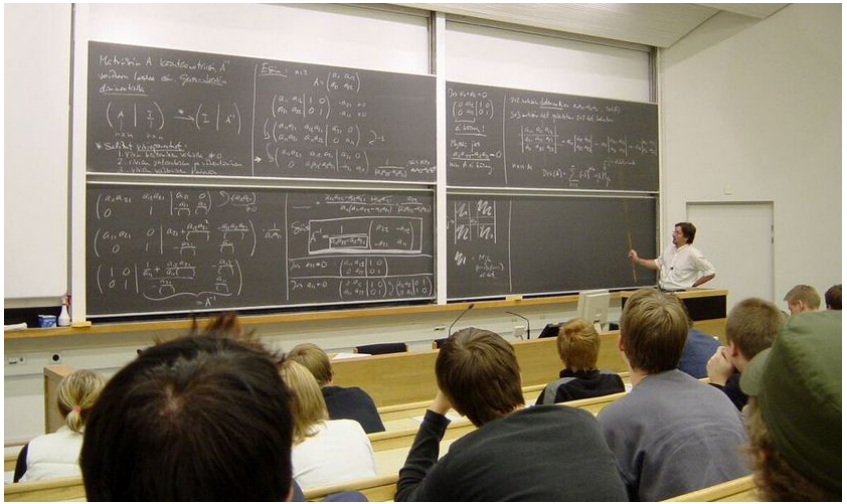
"Esta formulación inicial resultaba ambigua, ya que en ese momento no existía una definición matemática estricta sobre qué constituía exactamente un 'problema' ni qué era un 'procedimiento mecánico' (esa serie finita de pasos)."



David Hilbert

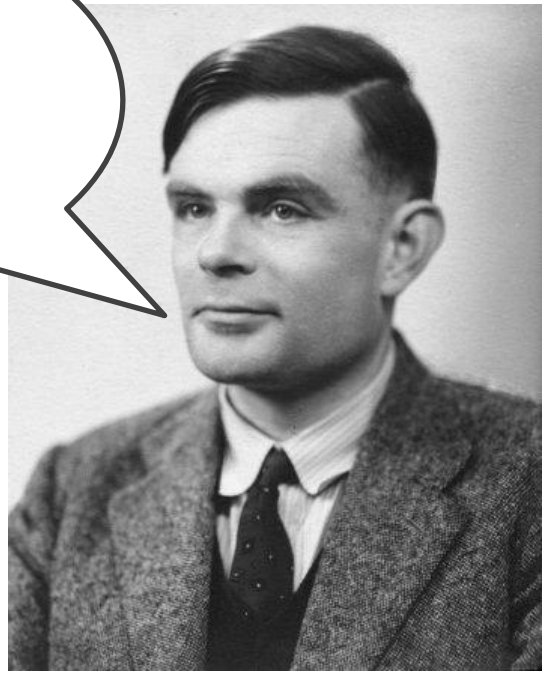
Fuente : Desconocido - Possibly Reid, Constance (1970) *Hilbert*. Berlin, Heidelberg: Springer Berlin Heidelberg Imprint Springer

Máquinas de Turing



Máquina de Turing

Hola, soy Alan Turing, en 1936 definí el concepto de algoritmo y propuse una máquina para computarlos



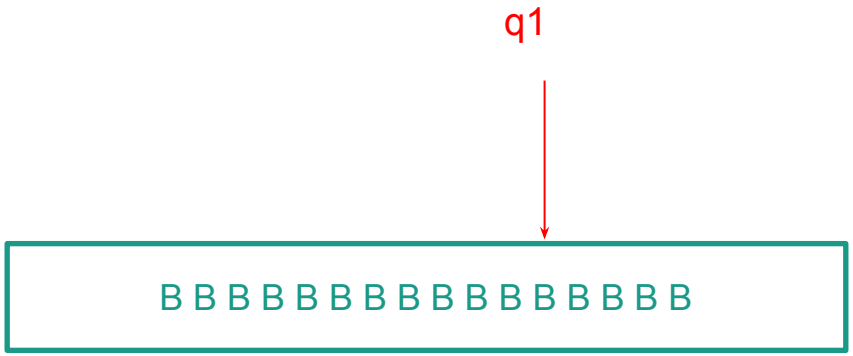


Máquina de Turing

Una cinta infinita y un cabezal que lee/escribe de la cinta.

Operaciones:

- Leer la cinta
- Escribir la cinta
- Mover la cinta a la izquierda
- Mover la cinta a la derecha
- Cambiar de estado
- Finalizar



Máquina de Turing

Una cinta infinita y un cabezal que lee/escribe de la cinta.

Las instrucciones tienen el formato:

(Qi, Li, Es, Ms, Qs)

Qi: estado al inicio del paso i
Li: Lectura de la cinta en el paso i
Ei: Escritura de la cinta en el paso i
Mi: Movimiento del cabezal en el paso i
Qs: Cambio de estado en el paso i

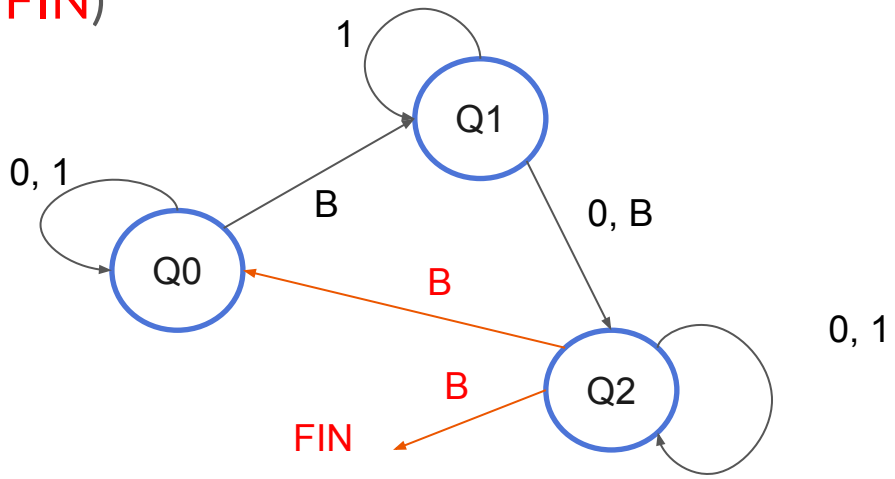
Máquina de Turing

¿Qué hace esta máquina?

(q0, 0,,D,), (q0, 1, ,D,), (q0, B, , , q1)

(q1, 0,1,,q2), (q1, 1, 0, I,), (q1, B, 1, , q2)

(q2, 0, , I,), (q2, 1, , I,), (q2, B, , D, q0/FIN)



Máquina de Turing

Pero…, ¿¿por qué hablamos de máquinas de Turing o de modelos tan limitados??, yo quiero mi lenguaje!











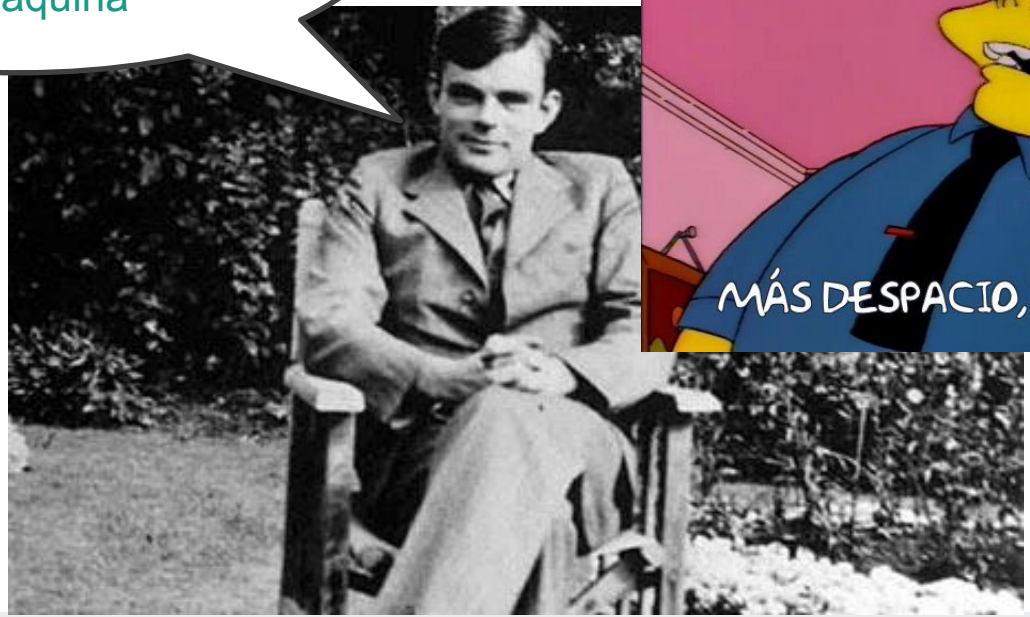
Máquina de Turing

Tranquilo!!, todo lo que hace un lenguaje moderno lo puede hacer mi máquina



Máquina de Turing

Tranquilo!!, todo lo que hace un lenguaje moderno lo puede hacer mi máquina





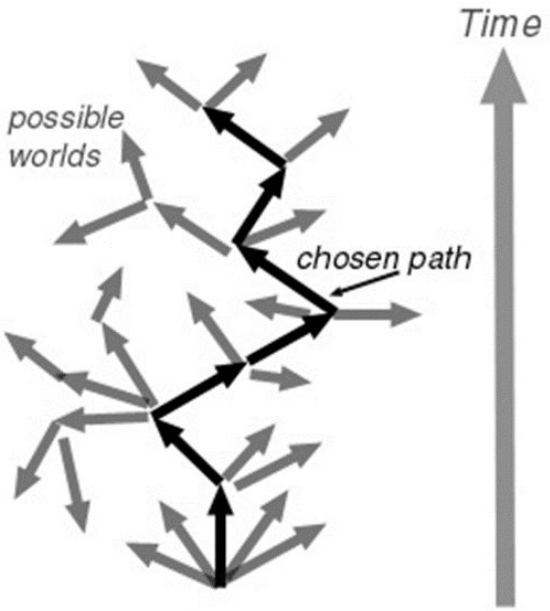
Máquina de Turing

- Se puede hacer un programa cuya salida sea el propio programa.
- Puede ser muy rápida, pero en cada instante tiene un único estado posible:
  - Es una máquina de Turing **determinística**



Máquina de Turing

- Una máquina de Turing **no determinística** puede replicarse y generar tantas copias de sí misma como sean necesarias.
- Multithreading EN SERIO.
- La MD tiene un hilo de procesamiento, la MND tiene un árbol.
- Necesito un procesador infinito.



Problemas de decisión

- Cada problema de optimización tiene asociado un problema de decisión, cuya respuesta es SI/NO.
- Por ejemplo, el problema del viajante en un grafo G, se convierte en la pregunta ¿Existe algún ciclo hamiltoniano en el grafo G que tenga una longitud menor a k?
- Ante cada respuesta, podemos ir modificando los valores de k hasta encontrar el óptimo.



Reducciones



Créditos: Débora Bonenberg

Reducciones

- ¿Qué problema es más difícil?
- ¿Ordenar una lista o devolver el mayor de una lista?

```
1
2
3  def mayor(v1[]):
4      ordenar(v1)
5      r = v1[0]
6      return(r)
7
8
9
```

```
1
2  def ordenar (v1[]):
3      while (len(v1[])>0):
4          a=mayor(v1)
5          borrar(v1,a)
6          v2.append(a)
7      return(v2)
8
9
```

- Si tenemos un programa que resuelve un problema, con un par de agregados, podemos resolver el otro.
- Encontrar el mayor se puede reducir a ordenar la lista, y ordenar una lista se puede reducir a encontrar el mayor.
- Podemos decir que uno de los problemas no es más difícil que el otro.

Reducciones (ahora sí)

- Si puedo transformar un problema Y en otro X usando operaciones de complejidad polinomial, estoy reduciendo (polinomialmente) Y a X:

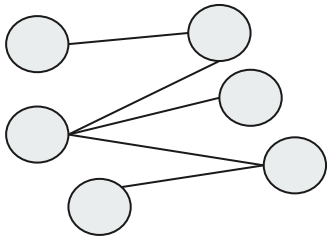
$$Y \leq_P X$$

Reducciones: Ejemplo 1

Bipartite Matching $\leq_P$ Redes de Flujo

BM → Transformación Entrada → RF → Transformación Salida → Solución BM

Grafo inicial

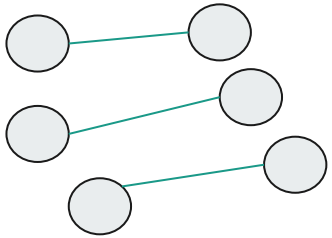


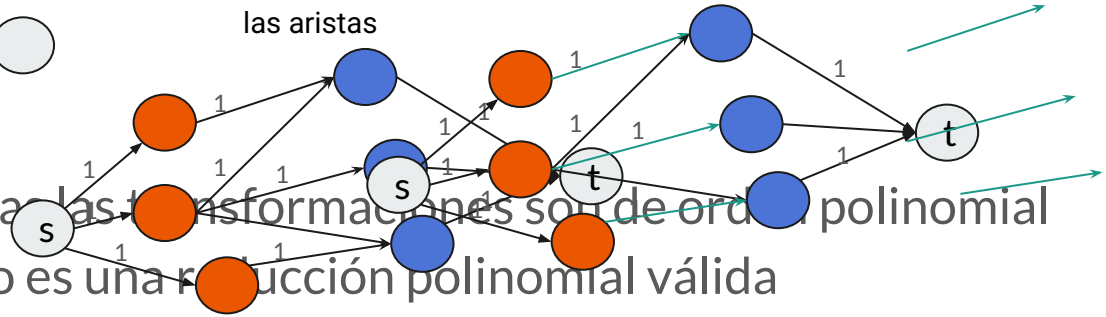
- Agregar vértices s y t
- Definir capacidad de las aristas

Resolver la red de flujo

A partir del grafo residual obtener las aristas con capacidad saturada

Solución del Bipartite Matching



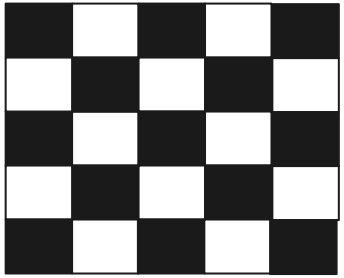


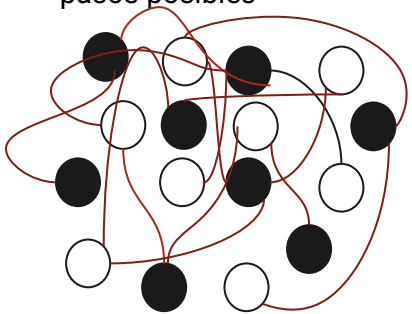
- Toda las transformaciones son de orden polinomial
- Esto es una reducción polinomial válida

Reducciones: Ejemplo 2

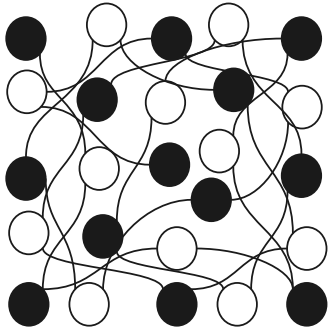
Problema del Caballo $\leq_P$ Camino Hamiltoniano

PC	Transformación Entrada	CH	Transformación Salida	PC
Tablero nxn	Crear el grafo con los pasos posibles	Resolver el Camino Hamiltoniano	No hace falta transformación	Solución del Problema del Caballo La lista de posiciones (correspondiente a aristas)





Faltan aristas pero se entiende



Reducciones: Ejemplo 2 (cont.)

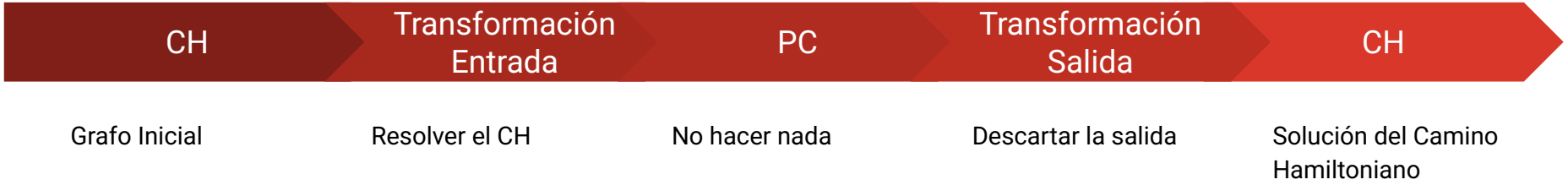
¿Se puede reducir el problema del Camino Hamiltoniano al Problema del Caballo?

- ¿Cómo transformaría un grafo arbitrario en un tablero de ajedrez?
- No se puede: porque el problema del Camino Hamiltoniano es más general que el Problema del Caballo

Un problema más específico se puede reducir a uno más general

Reducciones: Ejemplo 2 (cont.)

Camino Hamiltoniano $\leq_P$ Problema del Caballo



¿Y esto sirve para algo?

- Claramente no: porque resolver el Camino Hamiltoniano no es de tiempo polinomial

Reducciones: Ejemplo 3

Búsqueda Ordenada $\leq_P$ Búsqueda Desordenada



Array Ordenado	No hace falta hacer nada	Resolver la búsqueda suponiendo que la entrada está desordenada	No hace falta transformación	Solución de la Búsqueda Ordenada

Reducciones: Ejemplo 3 (ahora al revés)

Búsqueda Desordenada $\leq_P$ Búsqueda Ordenada



Array desordenado	Ordenar el array: O(n log(n))	Resolver la búsqueda suponiendo que la entrada está ordenada	No hace falta transformación	Solución de la Búsqueda Desordenada

¿Qué ganamos con esto?

- Ambos problemas (BD y BO) son de tiempo polinomial
- No ganamos nada

Reducir un problema de tiempo polinomial en otro de tiempo polinomial no agrega valor

Reducciones: Ejemplo 4

Búsqueda del Máximo $\leq_P$ Búsqueda de un Elemento



BM	Transformación Entrada	BE	Transformación Salida	BM
Array de Elementos (suponer que tienen valores enteros positivos)	Iterar desde 2^{bits} hasta 0 (bits=precisión del tipo de datos), llamando a BE para ver si encuentra ese valor	Resolver la búsqueda	No hace falta transformación	Solución de la Búsqueda del Máximo

¿Cuál es el problema de esta reducción?

- No es polinomial! Es exponencial!

  Una reducción no polinomial sólo empeora las cosas: no nos sirve

Reducciones: Ejemplo 5

Problema SAT:

- Tenemos un conjunto de n variables booleanas $x_1, \ldots, x_n$
- Cada una puede tomar el valor 1 o 0 (*true* o *false*)
- Un *término* es una variable x_i o su negación $\neg x_i$
- Una *cláusula* es una *disjunción* de varios *términos*: $x_1 \vee \neg x_2 \vee x_3 \vee x_4$
- Una *asignación* v satisface una *cláusula* si los valores de las variables hacen que la *cláusula* sea verdadera
- Una *asignación* v satisface una *colección de cláusulas* si los valores de las variables hacen que todas las *cláusulas* sean verdaderas

¿Existe una asignación v de las variables que satisfaga todas las cláusulas?

Reducciones: Ejemplo 5 (cont.)

Ejemplo:

$(x_1 \vee \neg x_2), (\neg x_1 \vee \neg x_3), (x_2 \vee \neg x_3)$

- Una asignación v donde $x_1=1$, $x_2=1$, $x_3=1$ no satisface la colección de cláusulas
- Una asignación v donde $x_1=1$, $x_2=0$, $x_3=0$ satisface la colección de cláusulas

Esta instancia de SAT tiene solución

Reducciones: Ejemplo 5 (cont.)

Variante: 3-SAT

- Es lo mismo que SAT, pero con cláusulas que tienen exactamente 3 términos

Ejemplo:

$(x_1 \vee \neg x_2 \vee \neg x_3), (\neg x_1 \vee x_2 \vee \neg x_3), (x_2 \vee \neg x_3 \vee \neg x_4)$

Reducciones: Ejemplo 5 (cont.)

$3\text{-SAT} \leq_P \text{SAT}$



Lista inicial de cláusulas y variables	No hace falta transformación	Resolver el Problema SAT	No hace falta transformación	Solución de 3-SAT

Reducciones: Ejemplo 5 (cont.)

¿Se podrá reducir SAT en 3-SAT ?

Sí, con un poco de álgebra de Boole



Lista inicial de cláusulas y variables

- Transformar cláusulas $(x \vee y)$ en $(x \vee y \vee z), (x \vee y \vee \neg z)$
- Transformar cláusulas (x) en $(x \vee z_1 \vee z_2), (x \vee \neg z_1 \vee z_2), (x \vee z_1 \vee \neg z_2)$... (el resto de las combinaciones)
- No modificar cláusulas con 3 términos
- Transformar cláusulas con más de 3 términos en $(x_1 \vee x_2 \vee z_1), (x_3 \vee \neg z_1 \vee z_2), (x_4 \vee \neg z_2 \vee z_3), \ldots, (x_{k-2} \vee \neg z_{k-4} \vee z_{k-3}), (x_{k-1} \vee l_k \vee \neg z_{k-3})$

Resolver el Problema 3-SAT

No hace falta transformación

Solución de SAT

Reducciones: Ejemplo 5 (cont.)

Mostramos que:

- SAT se puede reducir en 3-SAT
- 3-SAT se puede reducir en SAT

Ambos problemas son igual de “difíciles”
(más sobre esto en la próxima clase)

Reducciones: Conclusiones

- Las reducciones valiosas son las polinómicas.
- Una reducción de un problema polinomial en otro polinomial no es interesante.
- Las reducciones son transitivas:
  - Si $Z \leq_P Y$ y $Y \leq_P X$, entonces $Z \leq_P X$
- ¿Para qué sirven? Esto es Teoría de Algoritmos:
  - La parte teórica la vemos la clase que viene
  - La parte de algoritmos me sirve para convertir un problema que no sé resolver en uno que sí: Me piden resolver el Bipartite Matching y tengo una biblioteca que resuelve Redes de Flujo.



¡Muchas gracias!